

TALLER DE RESOLUCIÓN DE PROBLEMAS DE ALGORITMOS EN PSEUDOCÓDIGO Y DIAGRAMAS DE FLUJO

Evidencia: GA3-220501093-AA2-EV02

Aprendiz: Omar Enrique Carmona Figueroa

Programa de Formación: Análisis y Desarrollo de Software (ADSO)

Competencia: GA3-220501093 - Desarrollar soluciones algorítmicas a través de estructuras de control

Actividad de Aprendizaje: GA3-220501093-AA2

Instructor(a): [Nombre del Instructor]

Centro de Formación SENA
Regional Colombia
2026

1. Introducción

El desarrollo de software eficaz se fundamenta en la capacidad de razonamiento lógico y en la estructuración metódica de soluciones ante situaciones del mundo real. La algoritmia constituye la piedra angular de la programación, permitiendo descomponer problemas complejos en secuencias finitas, precisas y lógicas de instrucciones.

El presente documento contiene la solución formal correspondiente a la actividad de aprendizaje **GA3-220501093-AA2-EV02**. En esta entrega se analizan e implementan estructuras de control condicionales y cíclicas mediante lógica algorítmica expresada en pseudocódigo estándar (PSeInt). A través de tres problemas contextualizados —financieros, comerciales y matemáticos— se demuestra la aplicación de variables, acumulación de datos y toma de decisiones en el diseño de software.

1.1 Objetivos

Objetivo General:

Diseñar e implementar algoritmos eficientes en pseudocódigo y diagramas de flujo utilizando estructuras de control fundamentales para la resolución de problemas lógicos y computacionales.

Objetivos Específicos:

- Identificar con precisión los elementos de entrada, proceso y salida de cada problema planteado.
- Aplicar estructuras condicionales anidadas (Si-Sino) para la determinación de tablas de descuentos y reglas de negocio.
- Implementar estructuras repetitivas (Para) junto con operadores aritméticos y modulares para el procesamiento iterativo de funciones matemáticas.
- Validar la lógica de negocio a través del entorno PSeInt asegurando resultados exactos según las especificaciones técnicas requeridas.

Ficha del Programa: [Número de Ficha]

Duración Estimada: 14 Horas

Herramienta Utilizada: PSeInt / Diagramas A.

Estado de Evidencia: **COMPLETADO**

2. Desarrollo de la Actividad

2.1 Ejercicio 1: Cálculo de Valor Futuro (Interés Compuesto)

Enunciado: Una persona deposita hoy en el banco cierta cantidad de dinero con un interés del 2% mensual capitalizable mensualmente. Se requiere determinar el saldo acumulado al cabo de 5 años (60 meses).

Análisis de Variables:

- **Entrada:** valorPresente (Monto inicial consignado).
- **Proceso:** Aplica la fórmula $VF = VP \cdot (1 + i)^n$ donde $i = 0.02$ y $n = 60$.
- **Salida:** Impresión de valorPresente y valorFuturo.

Algoritmo en Pseudocódigo (PSeInt)

```
Algoritmo ValorFuturoBanco
  Definir valorPresente, valorFuturo, tasaInteres Como Real
  Definir meses Como Entero

  Escribir "Ingrese la cantidad de dinero a depositar hoy: "
  Leer valorPresente

  tasaInteres <- 0.02
  meses <- 60 // 5 años * 12 meses

  valorFuturo <- valorPresente * ((1 + tasaInteres) ^ meses)

  Escribir "Valor consignado: $", valorPresente
  Escribir "Valor futuro en 5 años: $", Redondear(valorFuturo)
FinAlgoritmo
```

2.2 Ejercicio 2: Descuento en Frutería

Enunciado: Calcular el total a pagar por la compra de manzanas a \$4.200/kg aplicando una escala progresiva de descuentos según el volumen en kilos comprados.

Rango de Kilos (K)	Descuento Aplicable
0.00 – 2.00 kg	0%
2.01 – 5.00 kg	10%
5.01 – 10.00 kg	15%
Más de 10.00 kg	20%

Algoritmo en Pseudocódigo (PSeInt)

```
Algoritmo DescuentoFruteria
  Definir kilos, precioKilo, subtotal, porcentajeDescuento, descuento, totalPagar Como Real

  precioKilo <- 4200

  Escribir "Ingrese la cantidad de kilos de manzanas a comprar: "
  Leer kilos
```

```

subtotal <- kilos * precioKilo

Si kilos >= 0 Y kilos <= 2 Entonces
    porcentajeDescuento <- 0.0
Sino
    Si kilos > 2 Y kilos <= 5 Entonces
        porcentajeDescuento <- 0.10
    Sino
        Si kilos > 5 Y kilos <= 10 Entonces
            porcentajeDescuento <- 0.15
        Sino
            porcentajeDescuento <- 0.20
        FinSi
    FinSi
FinSi

descuento <- subtotal * porcentajeDescuento
totalPagar <- subtotal - descuento

Escribir "La compra de ", kilos, " kilos tiene un valor de $", subtotal, ", pero tiene un
descuento por valor de $", descuento, ", por lo tanto, el valor a pagar es: $", totalPagar
FinAlgoritmo

```

2.3 Ejercicio 3: Evaluación de la Función $Y = x^2 - 2x$

Enunciado: Evaluar la función $Y = x^2 - 2x$ para valores enteros de x entre 1 y 10. Se debe calcular la suma total de Y , filtrar los múltiplos de 3 y los valores cuyo último dígito sea 5, junto con sus respectivas sumatorias acumuladas.

Algoritmo en Pseudocódigo (PSeInt)

```

Algoritmo EvaluacionFuncion
    Definir x, Y, sumaTotal, sumaMultiplos3, sumaTerminaCinco Como Entero

    sumaTotal <- 0
    sumaMultiplos3 <- 0
    sumaTerminaCinco <- 0

    Escribir "Valores de Y múltiplos de 3:"
    Para x <- 1 Hasta 10 Hacer
        Y <- (x ^ 2) - (2 * x)
        sumaTotal <- sumaTotal + Y

        Si Y MOD 3 = 0 Y Y <> 0 Entonces
            Escribir Y
            sumaMultiplos3 <- sumaMultiplos3 + Y
        FinSi
    FinPara

    Escribir "-----"
    Escribir "Valores de Y cuyo último dígito es 5:"
    Para x <- 1 Hasta 10 Hacer

```

```

Y <- (x ^ 2) - (2 * x)

Si Y MOD 10 = 5 Entonces
    Escribir Y
    sumaTerminaCinco <- sumaTerminaCinco + Y
FinSi
FinPara

Escribir "-----"
Escribir "Suma de todos los valores de Y: ", sumaTotal
Escribir "Suma de los valores de Y múltiplos de 3: ", sumaMultiplos3
Escribir "Suma de los valores de Y cuyo último dígito es 5: ", sumaTerminaCinco
FinAlgoritmo

```

3. Conclusiones

- La aplicación de diagramas de flujo y pseudocódigo permite abstraer la lógica del problema antes de la codificación en un lenguaje de programación formal, reduciendo errores semánticos y de sintaxis.
- Las estructuras condicionales compuestas y anidadas resultan fundamentales para modelar reglas de negocio flexibles, como sistemas tarifarios o escalas de descuento comerciales.
- El uso adecuado de ciclos de repetición combinados con operadores de acumulación y evaluación de residuos (MOD) optimiza la solución computacional de problemas matemáticos e iterativos.